NeuroARC.jl: Typed Criterial Causation, Hyphal Grafting, and Epistemic Invariants in Grid Domain Logic

Hunter Kinder
hkinder@stlteach.org
NeuroARC Autonomous Agent Research Group

November 2026

Abstract

Search-based solvers for the Abstraction and Reasoning Corpus (ARC) fail in three recurrent ways: multi-step compositions crash at runtime when intermediate grid types disagree; candidate verifiers return numeric sentinels (`Inf`, `-1.0`) that leak into beam scores; and untestable invariants are silently treated as passed or failed. We present `NeuroARC.jl`, a zero-dependency Julia package that addresses these failure modes through three typed layers: (1) a constructive evidence calculus (`TransjectionCalculus`) that separates operational three-way verdicts (`Accepted`, `Rejected`, `Incomplete`) from sealed proof and refutation certificates (`Witness`, `Refutation`, `Unresolved`) verified at construction time; (2) a preflight type-checking contract (`HyphalGraftingContract`) that rejects incompatible operator chains via `Out(A) <: In(B)` before executing any grid transformation; and (3) a deterministic candidate verifier combining exact training replay, informative leave-one-out parameter refitting, and a three-valued equivariance test under the eight symmetries of the square (D_4, order 8) and background-preserving color permutations. The package is verified across 652 unit-test assertions and enforces a four-level scientific achievement taxonomy separating implemented interfaces from experimentally supported claims.

Keywords: Abstraction and Reasoning Corpus; program synthesis; test-time training; criterial causation; constructive verification; typed composition; Julia.

1 Introduction and Epistemic Architecture Boundaries

The Abstraction and Reasoning Corpus (ARC) challenges current artificial intelligence architectures by requiring broad general intelligence across discrete grid environments [2]. Typical approaches rely on large language model code generation, brute-force domain specific language (DSL) enumeration, or end-to-end neural policy search. However, existing implementations often suffer from three systemic defects: (1) lack of type safety during dynamic program composition, leading to silent runtime type coercions or uninformative exceptions; (2) use of numerical sentinel values like `Inf` or `-1.0` to signal candidate failure, which frequently leak into numerical optimizers and distort beam search scoring; and (3) conflation of distinct epistemic categories, wherein speculative search heuristics are mislabeled as verified mathematical theorems.

To resolve these defects, we present `NeuroARC.jl`, a zero-dependency Julia package that formalizes the verification and typed-composition contract motivated by our ARC Prize 2026 competition runs. A central design principle of our research program is strict epistemological stratification across three distinct research tracks:

- **Track 1 (Kaggle ARC-AGI-2 Submission Track)**: The Python test-time training (NVARC) and symbolic sidecar submission pipeline evaluated on the ARC Prize 2026 semi-private leaderboard (`31.94` and `29.72` under a triple-pass schedule; `30.28` on S1, `28.47` on the triangulated replicate, and also `28.47` on S5 across three sidecar-enabled runs). Track 1 runs in Python on Kaggle's offline four-L4 GPU environment within the twelve-hour wall-clock limit; its failure audit motivated the formal contracts in Track 3.
- **Track 2 (Coevolutionary Aesthetics and Interface Theory Track)**: Kept rigorously minimalist, conservative, and focused exclusively on whether a coevolved aesthetic preference relation [10] escapes the 2006 Scrambling Theorem or is carried by a functional bijection [7, 8]. No ARC benchmarks, prompt engineering tokens, or ungrounded algorithmic operators enter Track 2.
- **Track 3 (NeuroARC.jl Formal Verification and Composition Track)**: The standalone Julia reference package for criterial causation, constructive evidence certificates, preflight typed composition (`HyphalGraftingContract`), leave-one-out verification, and pedagogical scaffolding.

By maintaining this separation, `NeuroARC.jl` formalizes candidate verification and operator composition as machine-checked computational contracts without conflating the standalone Julia formalization with the Python leaderboard submission notebooks or foundational perceptual theorems.

2 Tri-Stage Criterial Architecture and Logic Gates

Following Peter Tse's neuro-cognitive model of criterial causation [11], neural selection is modeled as informational gating over dynamic synaptic weights. In `NeuroARC.jl`, this process is operationalized as a tri-stage temporal architecture:

1. **Stage t_1 (Setting of Informational Criteria)**: The controller configures acceptance thresholds $\boldsymbol{\tau}$, projection operators $\Pi_{\mathcal{S}}$, and assigns initial computational token budgets based on generative entrenchment [12].
2. **Stage t_2 (Input Arrival and Informational Gating)**: Candidate transformations or sensory grid states $x \in \mathcal{X}$ are evaluated across a composition of criterial logic gates:
   - **Threshold Gate**: $g(x) = \mathbf{1}\{c(x) \geq \tau\}$
   - **Bandpass Gate**: $g(x) = \mathbf{1}\{\tau_{\min} \leq c(x) \leq \tau_{\max}\}$
   - **Subtype Gate**: $g(A, B) = \mathbf{1}\{\mathrm{Out}(A) <: \mathrm{In}(B)\}$ (evaluated with zero executions of node functions)
   - **Conjunction Gate (AND)**: $G_{\wedge}(x) = \prod_{k=1}^{K} \mathbf{1}\{c_k(x) \geq \tau_k\}$
   - **Disjunction Gate (OR)**: $G_{\vee}(x) = \max_{k=1}^{K} \mathbf{1}\{c_k(x) \geq \tau_k\}$
   - **K-of-N Gate**: $G_{k/N}(x) = \mathbf{1}\left\{\sum_{m=1}^{N} \mathbf{1}\{c_m(x) \geq \tau_m\} \geq k\right\}$
3. **Stage t_3 (Downward Empowerment or Extinction)**:
   - When criteria are satisfied, the candidate is projected onto the invariant subspace $\Pi_{\mathcal{S}}(x)$ and rewarded with downstream compute allocation via a constructive witness verdict `Accepted(value, salient_features, score, detail)`.

- When criteria fail, execution is halted immediately with zero downstream budget consumption, emitting a re-checkable refutation certificate `Rejected(reason, detail, failed_gate_indices, failed_gates, partial_score, violation_certificates)`, where each certificate records `(gate_name, pair_index, cell_residual, violation_coord)`.
- When prerequisites or symmetry anchors are untestable, the system emits an auditable verdict `Incomplete(reason, detail, missing_prerequisites)`, recording the specific missing prerequisite symbols without silent fallback.

2.1 Operational Three-Way Policy versus Formal Constructive Proof Calculus

Because intuitionistic logic admits no finite-valued truth-table semantics [5], we explicitly distinguish an **operational three-way verification policy** (`Accepted`, `Rejected`, and `Incomplete` status enumeration at the `CriterialVerdict` layer) from a **formally constructive proof calculus** implemented as `EvidenceStatus` with `Witness{T, C}` (C <: `AbstractProofTerm`), `Refutation{T, C}` (C <: `AbstractRefutationTerm`), and `Unresolved{T}`. An unguarded `certificate::Any` field would allow `Witness(prop, nothing)` or `Witness(prop, true)` to masquerade as a proof term. Instead, `Witness{T, C}` and `Refutation{T, C}` constrain C <: `AbstractCertificate` and invoke `check_proof_term` and `check_refutation_term` inside their inner constructors at instantiation time. Generic fallback methods `check_proof_term(::Any, ::Any) = false` and `check_refutation_term(::Any, ::Any) = false` fail closed on any unverified certificate subtype. For `StructuredProofTerm` and `StructuredRefutationTerm`, the constructor check verifies the structural hash seal, proposition alignment, and internal consistency of the recorded fields (such as measured value versus threshold and margin arithmetic for threshold and bandpass gates, subtype relation `out_type` <: `in_type` for subtype gates, recursive child-certificate validity for composite gates, and an optional `recheck` closure when supplied by `evaluate_gate_constructive`), while `Unresolved{T}` requires non-empty `missing_prerequisites` symbols when neither certificate is available. Constructive gate evaluation (`evaluate_gate_constructive`) and Brouwer-Heyting-Kolmogorov (BHK) combinators (`conjunction_evidence` and `disjunction_evidence`) enforce the strict epistemic invariant that an unavailable prerequisite yields `Unresolved{T}` and is never coerced into a `Refutation{T}` counterexample.

To ground continuous-to-discrete sensitivity and constructive evidence semantics without ambiguity, we contrast three distinct mathematical universes for smooth or nilpotent infinitesimals and constructive logic:

1. **Synthetic Differential Geometry (SDG) in Smooth Toposes**: Moerdijk and Reyes [9] construct intuitionistic topos models governed by the Kock-Lawvere microaffineness axiom [1,9], where every function on the nilsquare infinitesimal object $D = \{d \in R : d^2 = 0\}$ is uniquely affine.

2. **Classical-Modal Interpretation of Smooth Infinitesimal Analysis**: Hellman and Shapiro [6] formulate a classical-modal reinterpretation of nilsquare identity, demonstrating how validity depends critically on internal versus external quantifier scope.

3. **Classical-Logic Fermat Reals**: Giordano [4] develops the ring of Fermat reals within strictly classical logic, proving that nilpotent infinitesimals do not automatically force intuitionistic logic and making the choice of mathematical universe mathematically consequential.

Crucially, the informal slogan that smooth infinitesimals are "things indistinguishable from zero" is not a substitute for the Kock-Lawvere axioms, exact internal logic ($\neg\neg(d = 0)$ without $d = 0$), and explicit quantifier scope. Finally, to prevent epistemic inflation between abstract mathematical interfaces and empirical results, `NeuroARC.jl` enforces a four-level scientific achievement taxonomy (`AchievementLevel`): *implemented interface* (`IMPLEMENTED_INTERFACE`), *verified numerical component* (`VERIFIED_NUMERICAL_COMPONENT`), *validated physical model* (`VALIDATED_PHYSICAL_MODEL`), and *experimentally supported empirical claim* (`EXPERIMENTALLY_SUPPORTED_CLAIM`).

Furthermore, the pedagogical scaffolding loop of Wood, Bruner, and Ross [13] is instantiated as a controller across six tutorial functions (Recruitment, Reduction in Degrees of Freedom, Direction Maintenance, Marking Critical Features, Frustration Control, and Demonstration), managing state transitions across `Focus`, `Frustration`, and `Panic` modes.

3 Hyphal Grafting and Deep Composition Contracts

Complex ARC reasoning rules rarely arise as atomic primitives; they are formed by composing modular operators such as connected component extractors, spatial transformers, and stencil stampers. We formalize this composition through the biological metaphor of *hyphal grafting*.

3.1 Static Preflight Subtype Checking

Let A and B denote two transformation nodes encapsulated within parametric types:

$$A :: \texttt{GraftNode}\{\mathrm{In}_A, \mathrm{Out}_A\}, \quad B :: \texttt{GraftNode}\{\mathrm{In}_B, \mathrm{Out}_B\} \tag{1}$$

Before connecting A and B, the system evaluates Julia's subtype lattice relation:

$$\mathrm{Out}_A <: \mathrm{In}_B \tag{2}$$

If this condition is violated, the edge is rejected during preflight validation with zero executions of the underlying functions $A.f$ or $B.f$. This resolves a significant source of latency in large search graphs by pruning invalid compositional branches before dispatch.

3.2 Runtime Intermediate Boundary Enforcement

Static subtyping guarantees that declared signatures are compatible; however, it does not guarantee that an arbitrary implementation $A.f$ strictly respects its declared return type for all edge cases. To prevent unverified types from propagating across intermediate pipeline boundaries, the composed execution wrapper is defined as:

$$(B \circ A)(x) = B(A(x)) \tag{3}$$

where each node call wrapper $(N)(x)$ enforces:

$$\texttt{Core.typeassert}(N.f(x), \mathrm{Out}_N) \tag{4}$$

This ensures that any intermediate contract violation immediately triggers an auditable `TypeError` at the exact boundary of failure, rather than corrupting subsequent pipeline stages.

4 Deterministic Verifier and Epistemic Invariants

A critical challenge in visual abstraction is solipsistic overfitting, where candidate search algorithms fit demonstrations via memorization rather than inferring generalizable rules. `NeuroARC.jl` implements an isolated verification harness:

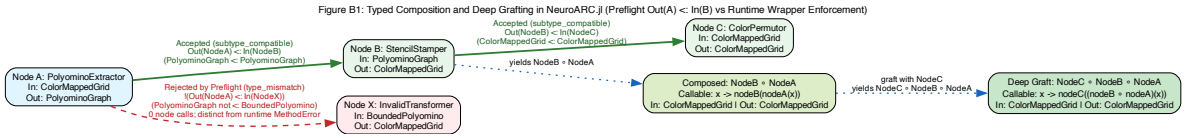


Figure 1: **Figure B1: Typed Composition and Deep Grafting Contract.** Illustrates preflight compatibility checking ($\mathrm{Out}(A) <: \mathrm{In}(B)$), typed verdicts (`Accepted` vs `Rejected`), associative three-stage deep grafting ($C \circ B \circ A$), and runtime intermediate boundary type assertions.

4.1 Verification Invariants

Before any candidate is admitted into the verified execution pool, it must satisfy three deterministic gates:

1. **Train Exact Reconstruction**: The candidate transformation must reconstruct 100% of cells across all training demonstration output grids ($Y_i = f(X_i)$) with maximum cell residual equal to 0.0.

2. **Informative Leave-One-Out (LOO) Evaluation**: Cross-validation across demonstration pairs is evaluated to detect parameter overfitting. When a rule contains free parameters, LOO folds are marked informative; parameter-free macros require verified cross-task firing to avoid single-task memorization.

3. **Equivariance Gating**: Where task semantics exhibit geometric invariance, the candidate must commute with the eight symmetries of the square (D_4, order 8) and background-preserving color permutations. If a candidate depends on directional or absolute spatial coordinates, equivariance is flagged as `untestable`, leading to an explicit `Incomplete` evaluation rather than a false positive certification.

The package exports seven `AbstractGridRule` subtypes: three rules with complete execution methods (`MarkerLineReflectionRule`, `ArcGeyserRule`, and `ArcGyrismRule`) and four typed extension stubs (`GeodesicBarrierFloodRule`, `CorridorWireTraceRule`, `PeriodicStampWithPriorityRule`, and `ExactCoverPolyominoRule`) that throw `RuleNotImplementedError` and evaluate to `:incomplete` under `verify_candidate` rather than being silently treated as training mismatches.

4.2 Epistemic Limits of Verification and Scientific Achievement Taxonomy

The verifier certifies compliance with observed demonstration contracts and structural invariants; it does not and cannot certify ground-truth accuracy on held-out test grids. Post-run test evaluation is conducted strictly through isolated scoring harnesses against quarantined held-out labels after solver processes terminate.

To enforce strict alignment between constructive proof terms and scientific claims, the verifier separates the operational status policy (`Accepted`, `Rejected`, `Incomplete`) from the formal constructive evidence hierarchy (`EvidenceStatus`: `Witness{T}`, `Refutation{T}`, and `Unresolved{T}`). Furthermore, `ScientificClaimAudit{T}` and `audit_scientific_claim_boundary` audit every scientific proposition against four ordered achievement levels:

1. **Implemented Interface** (`IMPLEMENTED_INTERFACE`): Typed structural signatures and compositional contracts are defined and type-checked.

2. **Verified Numerical Component** (`VERIFIED_NUMERICAL_COMPONENT`): Algorithmic and numerical invariants pass deterministic unit and leave-one-out verification suites with recheckable witness certificates.

3. **Validated Physical Model** (`VALIDATED_PHYSICAL_MODEL`): Physical or dynamical predictions are validated against independent reference benchmarks.

4. **Experimentally Supported Empirical Claim** (`EXPERIMENTALLY_SUPPORTED_CLAIM`): Empirical claims are supported by direct experimental replication and held-out benchmark evaluation.

If a proposition's `claimed_level` exceeds its `achieved_level` or its underlying evidence is a `Refutation{T}`, `audit_scientific_claim_boundary` immediately emits a `Refutation{T}` countercertificate. Conversely, if experimental prerequisites or independent verification records are not yet available, it emits `Unresolved{T}` recording the missing prerequisite symbols without coercing absent prerequisites into a refutation.

4.3 Structural Analogy: Symmetry Breaking and Single-Compartment Quarantine

The verifier's gate ordering is structurally analogous to directional recoil detectors in particle astrophysics [3], where enforcing geometric symmetry against a reference axis reduces the sample size required for background rejection across small event counts. Analogously, our leave-one-out and equivariance gates isolate each candidate evaluation inside a single compartment and record deterministic hash receipts so rejected or incomplete candidates never leak into downstream search.

5 Package Architecture and Zero-Dependency Core Isolation

To maintain high execution velocity and reproducibility on minimal environments, `NeuroARC.jl` enforces zero-dependency core isolation. The core library relies exclusively on the Julia standard library, implementing lightweight internal parsers and memory buffers without external binary dependencies.

5.1 Standard Julia Package Skeleton

The repository adheres to the standard Julia package layout:

- `Project.toml`: Declares package UUID (`a3f5a15a-93e1-4bd8-99e6-0563b78be05c`), version bounds on Julia (`1.10` to `1.13`), and conditional extension dependencies.
- `src/NeuroARC.jl`: Exposes public APIs for grid representations, component analysis, criterial calculus, and hyphal grafting contracts.
- `ext/`: Houses conditional package extensions that load only when external packages are present in the environment:
  - `NeuroARCMetalExt.jl`: GPU acceleration for Apple Silicon via `Metal.jl`.
  - `NeuroARCJACCExt.jl`: Cross-hardware portability via `JACC.jl`.
  - `NeuroARCTuringExt.jl`: Bayesian MCMC calibration via `Turing.jl`.

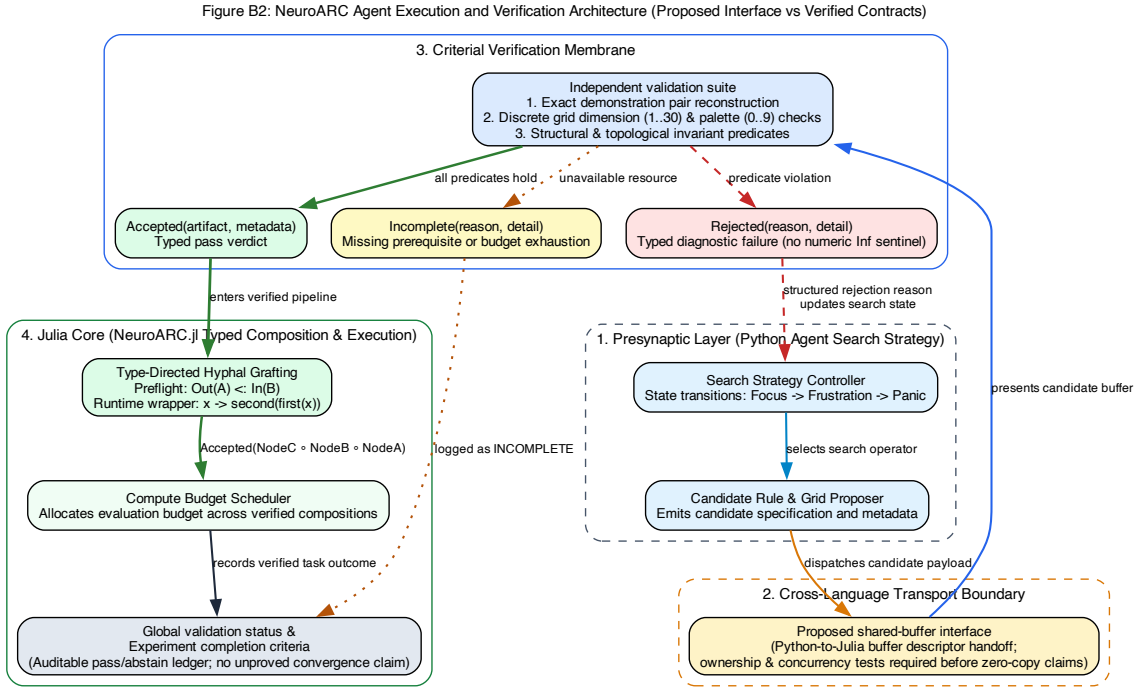


Figure 2: **Figure B2: NeuroARC Agent Execution and Verification Architecture.** Depicts the separation between presynaptic heuristic search, cross-language transport buffers, the criterial verification membrane, and the verified Julia composition core.

- `test/runtests.jl`: Comprehensive unit test suite executing 652 assertions across core types, criterial gates, constructive certificates, grafting contracts, and epistemic text cleanliness.

5.2 Continuous Integration and Clean Runners

The repository includes GitHub Actions CI workflows configured for clean runners on Ubuntu and macOS. These workflows verify that the package compiles and passes all unit tests without pre-installing non-standard dependencies, ensuring that clean installations remain fully functional.

6 Discussion, Competition Governance, and Repository Lifecycle

A central operational question addressed in this milestone is the relationship between our Python competition submission notebooks (Track 1) and the standalone `NeuroARC.jl` reference package (Track 3).

6.1 Empirical Context and Decontamination Governance

Three engineering constraints shaped the lifecycle from internal development to open-source release:

1. **Audit and Specification Invariants**: The anti-solipsism framework, the deterministic verifier threat model, and the constructive certificate calculus separate verified numerical components (`VERIFIED_NUMERICAL_COMPONENT`) from unverified empirical hypotheses.

2. **Kaggle ARC-AGI-2 Decontamination Governance**: In our audit of the upstream Python `dsl.py` module (164 top-level `FunctionDef` nodes), 73 definitions (72 unique runtime keys due to a duplicate `solve_b9e38dc0` definition) were task-keyed `solve_<8hex>` lookup tables rather than generalizable operators, leaving 91 reusable grid helpers. Exact-match regex stripping in the submitted clean run removed 71 runtime keys and leaked one renamed alias (`solve_9385bd28_generalized`) through a default-argument binding, collapsing accuracy from 39.17 on the public evaluation set (scored locally) to 0.00 on the hidden leaderboard. That failure motivated the constructive verification gates formalized here.

3. **Cross-Language Boundary between Python and Julia**: The Kaggle ARC-AGI-2 notebooks run in Python on Kaggle's offline four-L4 GPU environment within the twelve-hour wall-clock limit (reaching 31.94 and 29.72 under our triple-pass NVARC test-time training schedule, and 30.28 on S1, 28.47 on the triangulated replicate, and also 28.47 on S5 with the Python sidecar). `NeuroARC.jl` is the standalone Julia formalization of the sidecar's verification and typed-composition contract rather than the runtime engine executed inside the Kaggle submission kernels.

6.2 Open-Source Verification and Reproducibility

For the ARC Prize 2026 Paper Track release, the repository is packaged as a clean, zero-dependency Julia codebase verified by:

1. Deterministic execution of all 652 unit-test assertions via `Pkg.test()` across core types, constructive proof certificates, preflight grafting contracts, and text-cleanliness invariants.

2. Automated Graphviz and Tectonic compilation of the manuscript and architectural diagrams (`Figure B1` and `Figure B2`).

3. Public release alongside the companion Kaggle submission notebooks so reviewers can inspect both the empirical Python test-time training schedule and the formal Julia verification contract.

References

[1] John L Bell. *A Primer of Infinitesimal Analysis*. Cambridge University Press, 2nd edition, 2008.

[2] Francois Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[3] Andrzej K Drukier, Katherine Freese, Alejandro K Lopez, David N Spergel, Charles R Cantor, George M Church, and Takeshi Sano. New class of biological detectors for WIMPs. *International Journal of Modern Physics A*, 29(28):1443007, 2014. arXiv:1206.6809.

[4] Paolo Giordano. Infinitesimals without logic. *Russian Journal of Mathematical Physics*, 17(2):159 to 191, 2010.

[5] Kurt Gödel. Zum intuitionistischen aussagenkalkül. *Anzeiger der Akademie der Wissenschaften in Wien*, 69:65 to 66, 1932.

[6] Geoffrey Hellman and Stewart Shapiro. A classical-modal interpretation of smooth infinitesimal analysis. *The Review of Symbolic Logic*, 18(2):367 to 397, 2025.

[7] Donald D Hoffman. The scrambling theorem: A simple proof of the logical possibility of spectrum inversion. *Consciousness and Cognition*, 15(1):31 to 45, 2006.

[8] Donald D Hoffman, Manish Singh, and Chetan Prakash. The interface theory of perception. *Psychonomic Bulletin and Review*, 22(6):1480 to 1506, 2015.

[9] Ieke Moerdijk and Gonzalo E Reyes. *Models for Smooth Infinitesimal Analysis*. Springer, 1991.

[10] Richard O Prum. Coevolutionary aesthetics in human and biotic artworlds. *Biology and Philosophy*, 28(5):811 to 832, 2013.

[11] Peter U Tse. *The Neural Basis of Free Will: Criterial Causation*. MIT Press, 2013.

[12] William C Wimsatt. Developmental constraints, generative entrenchment, and the innate-acquired distinction. *Integrating Scientific Disciplines*, page 185 to 208, 1986.

[13] David Wood, Jerome S Bruner, and Gail Ross. The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2):89 to 100, 1976.